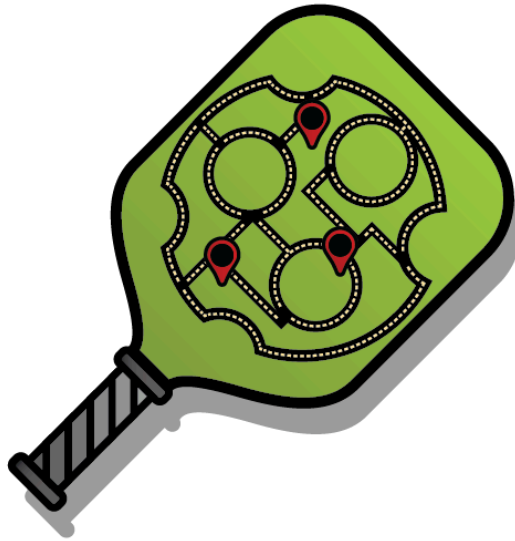




PaddleTraffic

Design Document

CS 4000

10/16/2023

Brensen Villegas, Camron Wilson, Ian McBride, Jacob Knowlton

Executive Summary

PaddleTraffic aims to provide a quick and easy way for pickleball players to find available pickleball courts in their area. It relies on people at the physical courts to report on how many groups are waiting for and using courts as well as using historical data. PickleTraffic uses both these factors to provide valuable insight into which court players should visit at any given moment.

Visiting PaddleTraffic's website (or downloadable mobile app version) starts players with a map view of courts near them and a convenient icon for each one to give a quick look at how busy they are. Tapping on any found court location allows for more details to be viewed, such as how many courts are at this location, exact number of players and groups waiting and playing, and insight into how this compares with previously recorded data. They are also able to connect with other players who are looking for someone to join their group and make a full team.

Proximity matchmaking and team formation for users is an additional feature provided through our site. Through locations of pickleball courts, players can meet up to play together through our app.

The core of PaddleTraffic will have a real-time map to show the activity of pickleball courts. This will be done by pulling data from our database and displaying the current status of courts to users based on the data, previous trends, and current submission to how busy the courts are. We will use front-end framework components and libraries to achieve the display of the map, which will work together with backend components for real-time data.

The source of our data will be provided via crowdsourcing. At locations of pickleball courts there will be QR codes put up that will link to our site. Using location services, users will be prompted to then enter data about the current status of that pickleball court. Questions that will prompt the users will include how many courts are currently available/occupied, and how many people are waiting if any at all. This will all be done through some sort of prompt screen that shows up to the user automatically. Crowdsourcing will also be used to know what other courts exist. Users will be able to submit requests to add new courts to the PaddleTraffic network. Questions will be asked such as, how many courts are at this location? Where is this location? Is it a paid or free court gathering? Are these indoor or outdoor courts? These will all be done through a separate menu that users can access on mobile or desktop through our website, which is the main application.

Background

Pickleball was named the fastest-growing sport in the United States by the SFIA, and is currently estimated to have over 5 million players worldwide. As the sport grows, it is becoming increasingly hard to find open courts to play on. For this reason, our app will help solve the problem of helping Pickleball players find an open court and tracking when each court is busy.

The primary users of our product will be pickleball players who want to find an available place to play pickleball and potentially join other players. Managers who are running recreational centers or other sports centers that have indoor or outdoor pickleball courts could also make great use of PickleTraffic to let their patrons know how busy they are.

Similar Existing Solutions

Pickleball players often show up to courts and don't know if it will be extremely busy, or completely empty. There are two well established solutions to this problem that are significant, Global Pickleball Network, and PickleHeads. Both of these are web applications that focus more on players meeting together to form teams. While both do have map features, they are lacking, and do not provide any information about the actual courts at the locations they suggest, and rather let players know a court exists at a location. We aim to focus more on finding real-time information about each court, not just where they are located. This provides a way to know which courts are busy at a glance, without having to travel to each one individually.

Global Pickleball Network: Focused on helping players organize events and find other players. However, this application is locked behind a paywall, has forced account creation, has no real-time traffic information, and is geared toward competitive players, whereas our application will be free to use and open to the public, and it will have real-time tracking information.

PickleHeads: Focused on helping players find courts and players near them. However, this application has no real-time traffic or reporting information, and there is no way to search based on your current location. As mentioned above, our application will have real-time traffic and historic information reporting, as well as location-services to let you automatically search for courts near you.

Required Technology

We are using a full stack web application with an integrated map showing real time data about how busy pickleball courts are, as well as a database to handle all persistent data. The following are the technologies we plan on using for our application:

- **Vue.js (Frontend Framework):** Using Vue.js as the frontend framework, we will be able to design a functional and good-looking user interface. From the frontend, users will be able to view a real-time map that shows how busy each Pickleball court location is, as well as view the previous busy times.
- **Django (Backend Framework):** With Django as our backend framework for handling requests and managing data, we will have a robust server that will be able to handle many simultaneous requests from users of our application.
- **MySQL (Database):** We will use MySQL as the database to store user reports and historical data, and help show trends about which courts are busy at which times.
- **Progressive Web Apps (Web Technology):** By building a progressive web app, we will make it possible for mobile users of this app to use this app as if it were a native app for their device.
- **AWS (Server Hosting):** By hosting our application on AWS, we will have a fast and robust system for handling requests, as well as easy scaling for any number of users.

Using most of the above technologies as frameworks, we will be building the frontend and backend components from scratch, as well as structuring the database from scratch. We will also leverage existing map libraries for the real-time map component for our application, with some customization to make it fit with our user interface and use case.

Software/Hardware Requirements

A user will only need access to a computer or mobile device that has a modern browser. This ensures that users can access the platform without the need for any additional installations or downloads. As our product is web-based, it emphasizes a low-friction experience, allowing users to simply connect to our website and begin their search effortlessly. We will also use location services for devices that are able. This will further reduce friction for our users and will provide a quick and easy way to find courts wherever our users want. Having a device with location services will provide the optimal experience for our users. Because of the web-based nature of our project, it will be highly compatible with a wide range of devices. In essence, our platform is designed with user convenience in mind, ensuring a seamless and efficient experience for all pickleball players.

Requirements Analysis

System Architecture

Frontend

Vue.js: This is a progressive JavaScript framework used for building user interfaces. Vue.js handles the frontend and offers features like state management, events, UI Components, and reusable interface elements (like buttons, modals, and other widgets) to ensure a consistent look and feel.

Page User Interface: This represents the visual and interactive part of the application where users interact directly. It includes event management, matchmaking, the map, and user login/registration.

Backend

Django: Django is a high-level Python web framework that allows for rapid development and a clean design. It would handle user account management which would store personal data, friendlists, map information and court data.

Matchmaking: Backend logic that supports the frontend matchmaking system.

Website Information: Content, news, updates, or any other static information presented to the user, as well as delivering the Vue.js project to the user.

Progressive Web Apps (PWAs): This technology will allow our web application to be installed on mobile devices and work offline, giving it a native app feel. Key components include a web manifest and mobile App Integration.

Hosting & Data

AWS Hosting: AWS (Amazon Web Services) will be used to host our application. AWS provides cloud services that allow applications to run on scalable and reliable infrastructure.

MySQL: MySQL is a relational database management system. We will use it to store accounts, map data, historical Data, and matchmaking data.

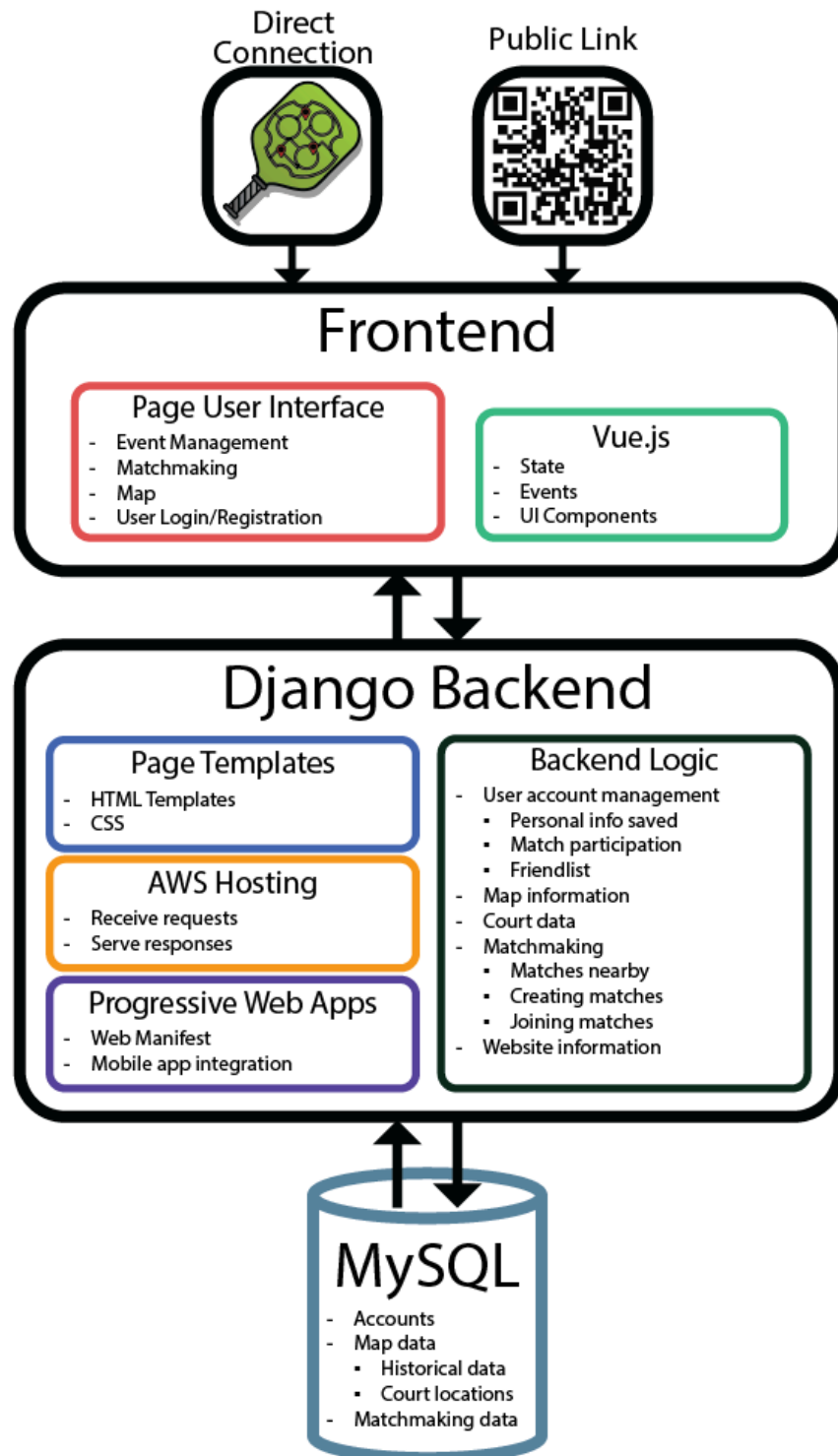


Figure 1: Web Application Architecture Flowchart

Personnel

Frontend Section

Vue.js Building of the Pages

Main Developer: Jake

- Jake has had experience with JavaScript and modern JS frameworks, including Vue.js and React. He has also previously built and deployed a few simple single-page applications using Vue.js.

Support Developer: Brensen

- Brensen has worked on web development projects where he utilized his skills in HTML, and CSS. As well as worked on android apps, windows forms, and similarly architected UI building softwares. This qualifies him to support the development of page elements.

Making Things Look Nice

Main Developer: Camron

- Camron has a keen eye for design and a rich portfolio that includes UI/UX design for web applications. He is proficient in CSS, SASS, and various design tools such as Adobe Illustrator and Figma. Camron's previous work in creating intuitive and aesthetically pleasing interfaces ensures that the application will not only function well but will also provide an excellent user experience.

Support Developer: Brensen

- Brensen has experience in crafting visually appealing app components. His ability to integrate form and function will complement Camron's design efforts.

Map Component

Main Developer: Ian

- Ian has experience with several interactive map-based projects. His knowledge of MapBox positions him to effectively take charge of the map component of the project.

Support Developer: Jake

- Jake has used MapBox to create a dynamic map in a previous web application, which is a framework we will likely use in our application.

Backend Section

Database Models/Design

Main Developer: Camron

- With a strong foundation in database theory and experience with SQL and NoSQL databases, that will scale with the application. Camron is well-equipped from prior work on web applications both in the backend communication and designing robust database models and architecture, assures that a solid data foundation will be established.

Support Developer: Ian

- Ian's experience spans implementing databases for web and mobile applications, equipping him with the practical knowledge to support the creation of efficient database schemas. He has developed good database practices from taking an elective class on databases.

The Django Backend

Main Developer: Brensen

- Brensen is proficient in Python. His familiarity with Django's class-based views and middleware from CS3550 makes him qualified for constructing a resilient and secure backend.

Support Developer: Ian

- Ian has used several backend frameworks and his understanding of python will be invaluable in supporting the development of the backend services.

AWS Deployment

Main Developer: Jake

- Jake has previously deployed a web application to another cloud based architecture, and is eager to learn AWS and help provide support to the team.

Support Developer: Camron

- Camron's experience in configuring and maintaining cloud-based environments, along with his understanding of CI/CD pipelines, complements Jake's skills and makes him a capable support for cloud deployment tasks.

System Features:

Rank 1: Infrastructure and Core Functionality:

- Core database tables/interactions for user information, court availability, and event management.
- Basic UI/UX for homepage, map view, and matchmaking pages to allow users to interact with the system effectively.

- Localhost implementation to ensure the application can run on a local machine before cloud deployment.
- Map system to show available courts near users of the application.
- Implementation of the primary algorithm for matchmaking and court scheduling to ensure effective session planning.
- Core networking capabilities to enable real-time updates and interactions between users and the application.

Rank 2: Planned Features:

- User/roles system for authentication and authorization to manage different user levels like players, event organizers, and club organizers.
- Expansion of the database to include real-time court status updates, event details, session schedules, and maintenance logs.
- Enhancement of the core UI/UX to allow for the creation of new events, scheduling of club sessions, and application maintenance.
- Publishing the application to the cloud for accessibility.
- API documentation and test suite for clear, external interactions with the application.
- Unit Test Suite for ensuring individual components work as expected.
- Continuous Integration/Continuous Deployment (CI/CD) pipelines for streamlined development and deployment processes.

Rank 3: Advanced Individual Features (Bells and Whistles):

- Integration with third-party services for event promotion, including social media platforms and email marketing tools.
- Development of an advanced analytics algorithm to compare current activity against historical data for court and event popularity insights.
- Building an administrative dashboard with data visualizations for application developers/managers to monitor system performance, user feedback, and error logs.
- Implementing a real-time chat feature for users to communicate with each other within the app for organizing games and discussing court conditions.
- Options for purchasing PickleCoin, a safe and secure investment opportunity.

Software Design

Software Engineering Tools

IDEs

- Visual Studio Code
- Pycharm

Main Libraries / Package Managers

- Poetry
- Node.js
- Vue.js
- Django
- DjangoRestFramework
- MapBox

Testing Plans and Tools

- Frontend testing with Vitest and end-to-end testing with Cypress

Bug and Issue Tracking

- Gitlab issues

Versioning

- Git

Software Engineering Techniques

Agile Methodology

We will employ an agile development process throughout our time working on this project. We will emphasize continuous improvement and adaptability as opposed to the more traditional waterfall model.

Code Reviews

Before merging any significant change to the main branch, at least one other member of the team will be required to sign off on the changes made. For large changes that could break old versions of the application, the whole team will be required to sign off on the changes.

Documentation

Wiki Pages

We are storing various important information about the project and its progress in a Gitlab managed set of Wiki pages. This consists of a general overview, notes and agendas from our weekly meetings, highlights of each team members' responsibilities, and more.

Read Me

We will have readme for the general and significant sections of our codebase to describe the overall architecture and functions of our code. This will ensure that we can have a quick and easy way to get an idea for what is in our code base from a "bird's eye" perspective.

In Code Commenting

Keeping simple summaries and explanations of more complex areas of code is integral to ensuring not only clean and readable code, but making sure that in the future, we can quickly remember what certain areas of code are doing and how to use them. This means that function definitions will be documented with header summaries, and any lines that we deem necessary to comment, will have further explanations in line.

Team Communication

Weekly Meetings

We will have weekly team meetings every Wednesday. In those meetings, we will go over the state of the project, road-blocks, goals for the next week, and team feedback. We will also schedule additional meetings as necessary to meet our project goals.

Daily Communication

We will primarily be using discord as our communication. When we need to assign tasks, we will do so primarily during our weekly meetings which will also be in discord or in person, but will also assign/re-assign tasks throughout the week as needed. This allows us to keep tabs on all of our progress and receive quicker feedback or information that is needed during the week.

Timeline

Alpha (Weeks 1-4)

	Brensen	Camron	Ian	Jacob
Week 1	Start framework for public and private event creation.	Initiate design for historical data collection interface.	Work on integrating location addition functionality.	Begin developing the user login system.
Week 2	Develop backend support for event management.	Enhance user interface for busy-times forecasting.	Refine location addition features.	Implement user registration functionality.
Week 3	Create UI elements for event details.	Start developing algorithms for busy-times forecasting.	Implement basic heat-map colors for map pins.	Test and debug user login and registration features.
Week 4	Integrate event creation with backend database.	Complete first iteration of busy-times forecasting tool.	Enhance map pins with interactive heat-map features.	Finalize user account management system.

Beta (Weeks 5-8)

	Brensen	Camron	Ian	Jacob
Week 5	Test and debug event creation and management features.	Optimize historical data collection methods.	Develop advanced features for map visualization.	Start integrating user feedback into account systems.
Week 6	Refine UI/UX of event management.	Begin scalability tests for forecasting algorithms.	Introduce more detailed heat-map color gradations.	Implement additional security measures for user accounts.
Week 7	Start work on notifications for events.	Enhance data analysis tools for better	Improve map interaction and usability.	Collaborate on integrating account features

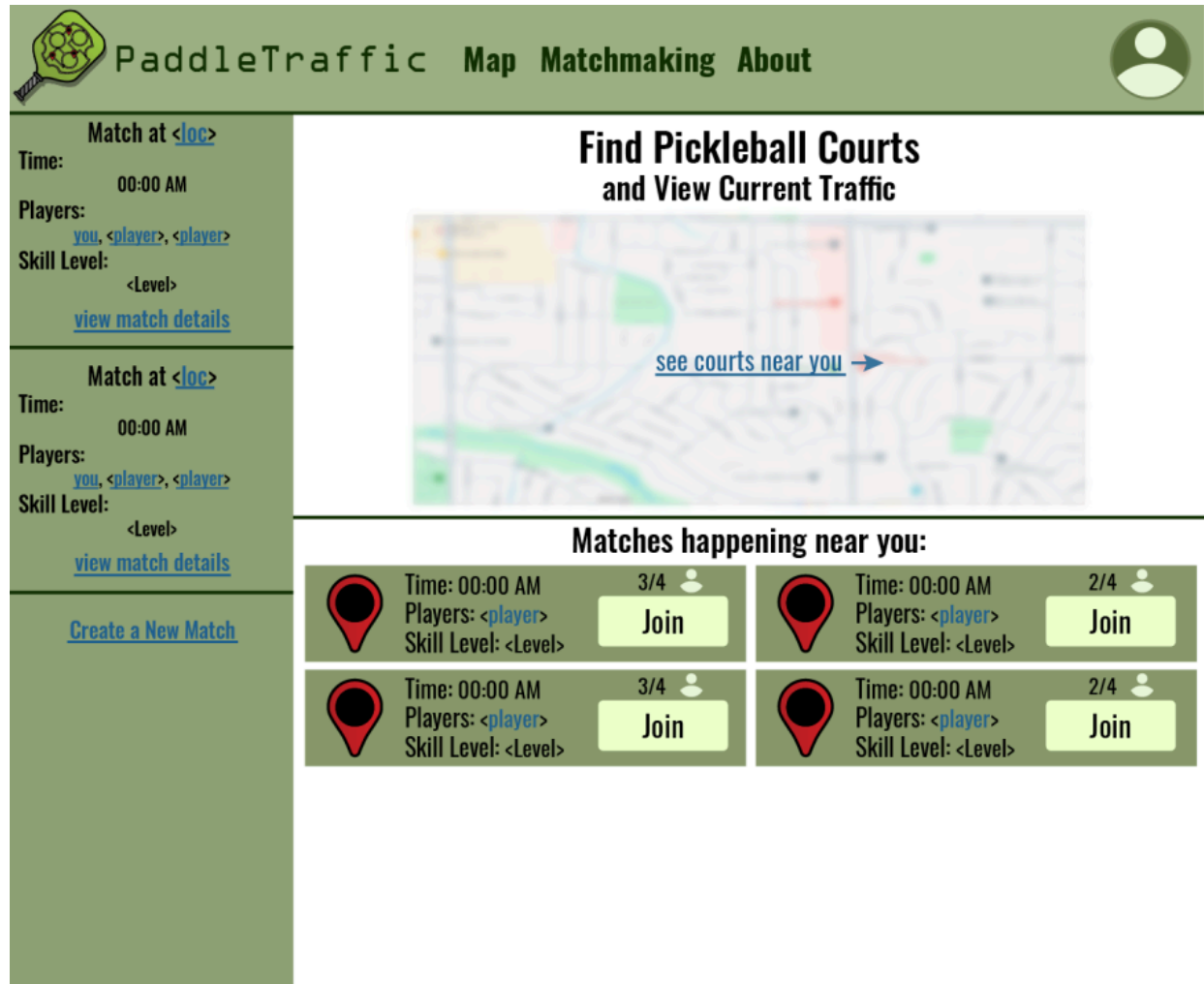
		forecasting accuracy.		with event management.
Week 8	Complete event notifications and start user testing.	Conduct comprehensive tests on forecasting tools.	Finalize all map visualization enhancements.	Finalize all account-related features and start documentation.

Final (Weeks 9-12)

	Brensen	Camron	Ian	Jacob
Week 9	Address any bugs in event management.	Begin final adjustments on forecasting based on beta tests.	Optimize map system for final deployment.	Work on final integration and system-wide testing.
Week 10	Enhance user experience based on feedback.	Finalize all features related to historical data and forecasting.	Ensure map system compatibility with different devices.	Prepare user documentation and support materials.
Week 11	Implement last-minute improvements to event features.	Review and optimize data storage solutions.	Conduct stress tests on map functionalities.	Lead final testing and debugging phase.
Week 12	Ensure seamless integration of all event-related features.	Prepare a report on data handling and forecasting accuracy.	Finalize and deploy the enhanced map system.	Final preparations for deployment.

Appendix A: UI Sketches

Home or Landing Page:



Map Page:


[PaddleTraffic](#)
[Map](#)
[Matchmaking](#)
[About](#)


Match at [<loc>](#)

Time: 00:00 AM

Players: [you](#), [<player>](#), [<player>](#)

Skill Level: [<Level>](#)

[view match details](#)

Match at [<loc>](#)

Time: 00:00 AM

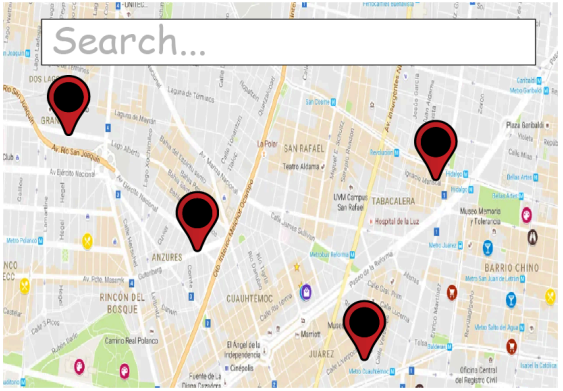
Players: [you](#), [<player>](#), [<player>](#)

Skill Level: [<Level>](#)

[view match details](#)

[Create a New Match](#)

Find Pickleball Courts and View Current Traffic



Filters and Map Legend go here



<Location Name>
 Courts: XX / YY
 Players Waiting: XX
 Est. Wait Time: XX Mins

Add Report

Create Match Here

Matchmaking Page:


[PaddleTraffic](#)
[Map](#)
[Matchmaking](#)
[About](#)


Match at [<loc>](#)

Time: 00:00 AM

Players: [you](#), [<player>](#), [<player>](#)

Skill Level: [<Level>](#)

[view match details](#)

Match at [<loc>](#)

Time: 00:00 AM

Players: [you](#), [<player>](#), [<player>](#)

Skill Level: [<Level>](#)

[view match details](#)

[Create a New Match](#)

Find and Create Events

Create a new Event:

Add players:

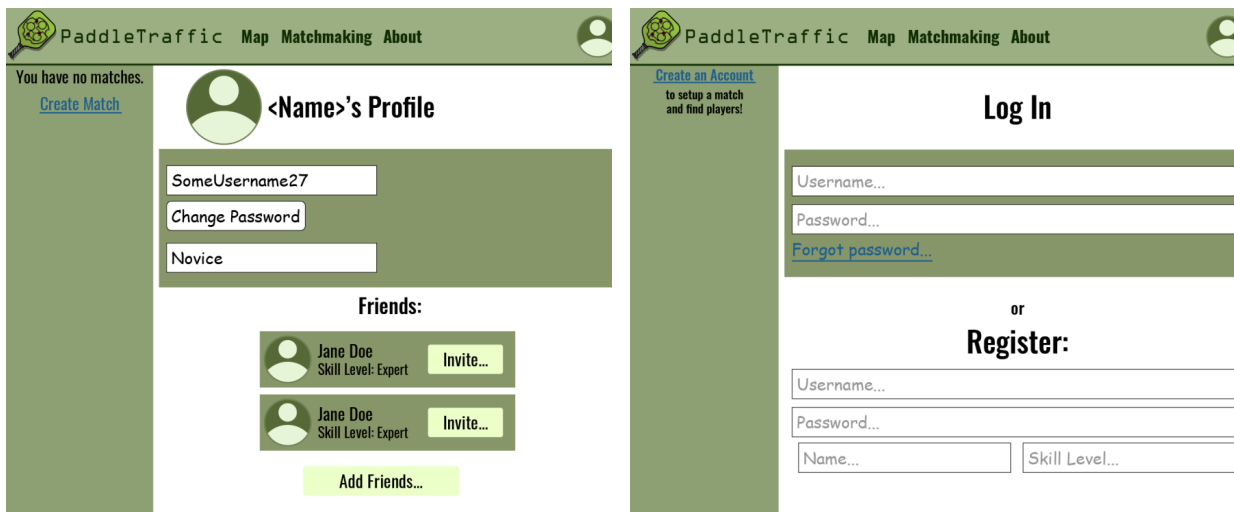
Games needing players:

 Time: 00:00 AM Players: <player> Skill Level: <Level> 2/4  Join	 Time: 00:00 AM Players: <player> Skill Level: <Level> 2/4  Join
 Time: 00:00 AM Players: <player> Skill Level: <Level> 2/4  Join	 Time: 00:00 AM Players: <player> Skill Level: <Level> 2/4  Join
 Time: 00:00 AM Players: <player> Skill Level: <Level> 2/4  Join	 Time: 00:00 AM Players: <player> Skill Level: <Level> 2/4  Join

About Page:



Profile Page (Logged in and Logged out):



Appendix B: Use Cases

Use Case 1: Finding Nearby Pickleball Courts

Preparer: Ian McBride

Actor/User: Pickleball Players

User Story: As a pickleball player, I want to use PaddleTraffic to find nearby pickleball courts and check their availability and additional team members to play with, so that I can plan my games effectively.

Course of Events:

1. Open the PaddleTraffic website.
2. View a map with icons representing nearby pickleball courts.
3. Tap on an icon to get detailed information about a specific court, including the number of available courts, the exact number of players and groups waiting, available slots for team members, and a comparison with historical data.
4. Decide which court to visit based on the provided information.

Exceptions/Alternatives: None

Related UI: *Home Page (pg 7), Map Page (pg 8), and Matchmaking Page (pg 8)*

Use Case 2: Real-time Court Status Updates

Preparer: Brensen Villegas

Actor/User: Paddleball Court Visitors

User Story: As a pickleball court visitor, I want to use PaddleTraffic to provide real-time updates on the status of the court I'm at, so that other players can see the current court availability and occupancy.

Course of Events:

1. Visit a pickleball court.
2. Use the PaddleTraffic app to report the current status of the court.

3. Enter and submit data such as the number of available/occupied courts and the number of people waiting.
4. The UI updates with the court status on the PaddleTraffic app interface.

Exceptions/Alternatives: None

Related UI: *Home Page (pg 7) and Map Page (pg 8)*

Use Case 3: Creating a New City Event

Preparer: Jacob Knowlton

Actor/User: City Event Organizer

User Story: As a city event organizer, I want to use the event management system to create a new city event, providing all the necessary details and logistics, so that I can efficiently plan and promote the event to the community.

Course of Events:

1. Log in to the application, and go to the event management system.
2. Access the "Create New Event" feature.
3. Enter event details such as event name, date, time, location, and description.
4. Specify event type, whether it's a tournament, clinic, city event, or unofficial meetup.
5. Review and confirm event details.
6. Publish the event to make it visible to the public.
7. Publish the event to social media and other channels to promote awareness.
8. Review and monitor event popularity.

Exceptions/Alternatives: If the event requires special permits, the system may guide the organizer to the relevant permit application process.

Related UI: *Map Page (pg 8), Matchmaking Page (pg 8), and Login Page (pg 9)*

Use Case 4: Scheduling Club Pickleball Sessions

Preparer: Brensen Villegas

Actor/User: Pickleball Club Organizer

User Story: As a pickleball club organizer, I want to use the club management system to schedule pickleball sessions, allowing club members to plan their attendance and ensuring smooth club operations.

Course of Events:

1. Log in to the event management system.
2. Access the "Schedule Sessions" feature.
3. Define the date, time, and duration of the upcoming pickleball session.
4. Specify the location or court where the session will take place.
5. Add any specific session notes or guidelines for club members.
6. Publish the session schedule to the club's calendar.
7. Notify club members about the newly scheduled session via email or in-app notifications.
8. Monitor the RSVPs and attendance for the session.
9. Update the session status or make any necessary changes if required.

Exceptions/Alternatives: If a session needs to be canceled or rescheduled due to unforeseen circumstances, the organizer can use the system to make those adjustments.

Related UI: *Map Page (pg 8), Matchmaking Page (pg 8), and Login Page (pg 9)*

Use Case 5: Application Maintenance and Monitoring

Preparer: Jacob Knowlton

Actor/User: Application Developer/Manager

User Story: As the developer/manager of the PaddleTraffic application, I want to efficiently maintain and monitor the application to ensure its stability, performance, and security.

Course of Events:

1. Access the PaddleTraffic application management dashboard.
2. Monitor the application's server and network performance.
3. Review error logs and identify any issues or anomalies.
4. Analyze user feedback and bug reports.
5. Plan and perform routine software updates and patches.
6. Implement security measures and monitor for potential vulnerabilities.
7. Monitor application scalability and resource usage.

8. Optimize the application code and database queries for performance.
9. Implement backup and disaster recovery procedures.

Exceptions/Alternatives: In the event of critical system failures or security breaches, the developer/manager may need to enact emergency procedures.

Related UI: *All Pages*

Revisions Appendix

- **Executive Summary:** No changes
- **Background:** No changes
- **Requirements Analysis:** No changes
- **Software Design:** No changes
- **Timeline:** Added section. Included timeline for all team members for next semester
- **Appendix A: UI Sketches:** Moved to end of document
- **Appendix B: Use Cases:** Moved to end of document
- **Revisions Appendix:** Added section